UDOO KEY DUAL-MCU UNIVERSAL ELECTRONICS INSTRUMENT

Definitive Gemini / Antigravity Project Master Specification & Execution Instructions

---

0. ROLE

You are the primary engineering agent responsible for **finishing the UDOO KEY electronics diagnostic instrument**.

Your job is not merely to analyze, explain, propose ideas, or generate theoretical architecture.

Your job is to:

1. establish the actual hardware truth,
2. reconcile previous audits and specifications,
3. inspect the existing repository,
4. inspect existing reference implementations,
5. identify real implementation gaps,
6. make the necessary code changes,
7. build all affected targets,
8. test each subsystem incrementally,
9. distinguish software tests from real hardware tests,
10. document unresolved hardware questions,
11. avoid unsupported capability claims,
12. and move the project toward a physically working instrument.

Do not repeatedly restart the architecture discussion.

Once something has been sufficiently verified, freeze it and build on it.

Do not replace working implementations merely because a different architecture appears theoretically cleaner.

---

1. PROJECT OBJECTIVE

Build a practical electronics diagnostic, measurement, protocol-analysis, and instrument-control system

using **one UDOO KEY board**.

The core board contains:

- RP2040
- ESP32-WROVER-E

No additional MCU or development board is part of the core instrument.

External passive/analog hardware, protection circuits, USB interface hardware, connectors, probes, and external test equipment may be used where required.

The instrument should combine the useful capabilities of:

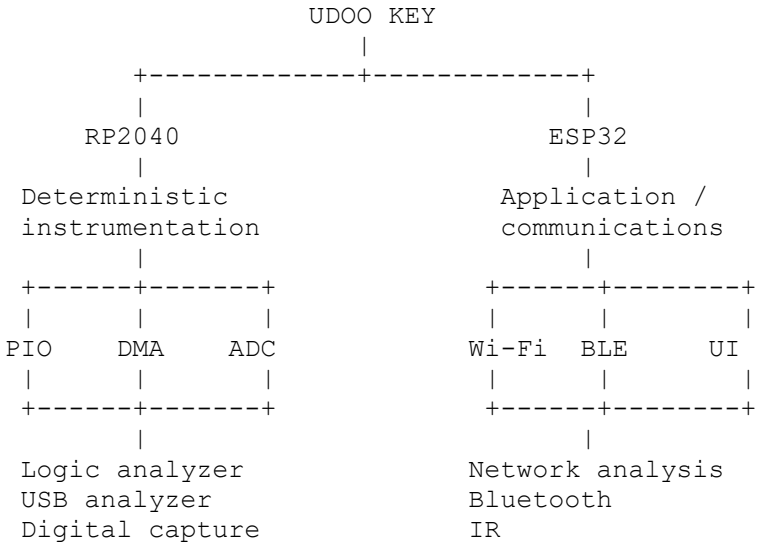
- compact logic analyzers,
- Bus Pirate-style protocol tools,
- protocol analyzers,
- USB analyzers,
- Wi-Fi/Bluetooth diagnostic tools,
- IR/nRF24 tools,
- JTAG/SWD tools,
- digital measurement instruments,
- and external oscilloscope control.

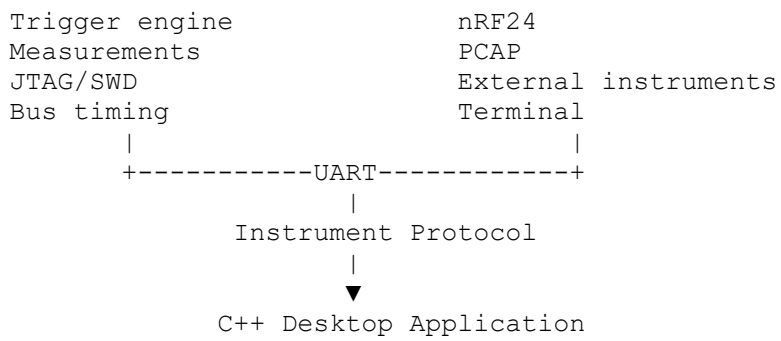
The project should remain realistic about what the UDOO KEY hardware can and cannot physically do.

---

2. CORE ARCHITECTURE

The architecture is:





The two MCUs are one instrument.

They are not independent products.

---

3. PRIMARY ENGINEERING PRINCIPLE

RP2040 = DETERMINISTIC INSTRUMENT ENGINE

The RP2040 owns anything requiring deterministic timing.

This includes:

- digital sampling,
- PIO,
- DMA,
- circular acquisition,
- hardware-assisted triggering,
- USB low/full-speed capture where supported,
- USB bit-level processing,
- timestamping,
- edge/event measurement,
- frequency measurement,
- pulse-width measurement,
- digital measurements,
- ADC acquisition where appropriate,
- external trigger input,
- prescaler control,
- capture metadata,
- deterministic bus engines,

- JTAG/SWD timing where appropriate.

The RP2040 must not depend on ESP32 scheduling for real-time acquisition.

The ESP32 must never introduce nondeterministic latency into the sampling path.

---

4. ESP32 = APPLICATION / COMMUNICATION / RF ENGINE

The ESP32 owns:

- displays,
- buttons,
- user interface,
- configuration,
- command handling,
- terminal,
- Wi-Fi,
- Bluetooth/BLE,
- network diagnostics,
- packet processing,
- PCAP generation,
- IR,
- nRF24,
- higher-level protocol analysis,
- external instrument communications where physically supported,
- data export,
- high-level USB host/device management where appropriate,
- protocol translation,
- desktop communication.

The ESP32 requests deterministic operations from the RP2040.

It does not perform the primary high-speed logic acquisition.

---

5. ARCHITECTURAL OWNERSHIP RULE

When deciding which MCU should implement a feature:

Prefer RP2040 when the feature requires:

- deterministic timing,
- high-rate GPIO sampling,
- PIO,
- DMA,
- exact pulse generation,
- exact pulse measurement,
- low-jitter triggering,
- USB low/full-speed bit capture,
- hardware-assisted digital protocols.

Prefer ESP32 when the feature requires:

- Wi-Fi,
- Bluetooth,
- networking,
- packet processing,
- filesystem/export,
- displays,
- terminal,
- application-level protocol handling,
- external instrument control,
- higher-level protocol decoding.

Shared features

Some systems cross both MCUs.

Examples:

```
USB:
physical capture → RP2040
protocol decode → RP2040/ESP32
UI → ESP32
desktop visualization → C++

Wi-Fi:
```

```
radio → ESP32
packet processing → ESP32
UI → ESP32
desktop analysis → C++

Logic Analyzer:
sampling → RP2040
capture transport → ESP32
UI → ESP32
analysis → desktop

HP 54520A:
physical interface → appropriate external interface hardware
instrument protocol → ESP32
waveform processing → ESP32/desktop
UI → ESP32/C++
```

---

6. HARDWARE AUTHORITY

Hardware truth must follow this hierarchy.

Authority 1 — Actual UDOO KEY schematic

The schematic is authoritative for:

- electrical connections,
- MCU-to-MCU connections,
- GPIO ownership,
- connectors,
- jumpers,
- onboard peripherals,
- LEDs,
- board control,
- internal signals,
- inaccessible signals.

Authority 2 — Official SECO UDOO KEY documentation

Use official documentation for:

- board specifications,
- connector pinouts,
- electrical limits,

- board variants,
- programming,
- documented functionality.

Authority 3 — Board-specific implementation references

Use the UDOO KEY Arduino reference and equivalent board-specific references to clarify:

- GPIO mappings,
- internal UART,
- LEDs,
- UEXT,
- SWD,
- board-specific implementation details.

Authority 4 — Existing project source

Determine:

- what already works,
- what is partially implemented,
- what conflicts,
- what is obsolete,
- what is dead code,
- what must be changed.

Authority 5 — MCU documentation

Use RP2040/ESP32 documentation for:

- PIO,
- DMA,
- timers,
- UART,
- SPI,
- I2C,
- ADC,
- USB capabilities,
- GPIO behavior,

- radio capabilities.

Generic MCU capability does **not** prove physical availability on the UDOO KEY.

---

7. ABSOLUTE GPIO RULE

Never confuse:

> MCU GPIO capability

with:

> physical accessibility on the UDOO KEY.

For every GPIO determine:

1. MCU,
2. GPIO number,
3. electrical function,
4. physical connection,
5. connector/pad/test point,
6. onboard peripheral ownership,
7. whether it can safely be repurposed,
8. project ownership,
9. verification state.

Use these classifications:

```
EXTERNAL_USER_GPIO
INTERNAL_MCU_CONNECTION
ONBOARD_PERIPHERAL
ONBOARD_LED
BOARD_CONTROL
SWD
RESERVED
UNUSED
UNKNOWN
REQUIRES_HARDWARE_VERIFICATION
```

Never call an unverified GPIO a "ghost pin."

---

8. CANONICAL RP2040 RESOURCE MAP

The following is a **project proposal**, not automatically verified hardware truth.

Verify against the actual board before freezing it.

```
GP0   internal UART RX       RESERVED
GP1   internal UART TX       RESERVED

GP2   Logic CH0              VERIFY
GP3   Logic CH1              VERIFY
GP4   Logic CH2              VERIFY
GP5   Logic CH3              VERIFY
GP6   Logic CH4              VERIFY
GP7   Logic CH5              VERIFY
GP8   Logic CH6              VERIFY
GP9   Logic CH7              VERIFY

GP10  External Trigger       VERIFY

GP11  TFT RESET              VERIFY
GP12  TFT DC                 VERIFY
GP13  TFT CS                 VERIFY

GP14  Prescaler Control      VERIFY

GP15  Button                 VERIFY
GP16  Button                 VERIFY
GP17  Button                 VERIFY

GP18  TFT SPI SCK            VERIFY
GP19  TFT SPI MOSI           VERIFY
GP20  TFT SPI MISO           VERIFY

GP21  Button                 VERIFY
GP22  Button                 VERIFY

GP23  Board/Internal         DO NOT ASSUME
GP24  Board/Internal         DO NOT ASSUME
GP25  Green LED              RESERVED

GP26  ADC0                   VERIFY
GP27  ADC1                   VERIFY
GP28  ADC2                   VERIFY

GP29  Board/Internal         DO NOT ASSUME
```

---

9. INTERNAL UART

Use the existing internal RP2040 ↔ ESP32 UART.

Proposed mapping:

```
RP2040 GP0 = UART RX
RP2040 GP1 = UART TX

ESP32 GPIO19 = UART TX
ESP32 GPIO22 = UART RX
```

Treat this as the preferred internal transport unless actual testing proves it inadequate.

Do not redesign the entire transport around external SPI without evidence.

The internal UART preserves external RP2040 GPIO resources.

---

10. TRANSPORT PROTOCOL

Create a transport-neutral binary protocol.

Do not use SPI-specific structures if UART is the actual transport.

Minimum packet structure:

```
MAGIC
VERSION
TYPE
FLAGS
SEQUENCE
PAYLOAD_LENGTH
PAYLOAD
CRC
```

The protocol must support:

```
PING
PONG
GET_STATUS
SET_CONFIG
START_CAPTURE
STOP_CAPTURE
GET_CAPTURE_INFO
GET_CAPTURE_DATA
TRIGGER
GET_MEASUREMENT
GET_ADC
ERROR
```

All packets must have:

- version,
- sequence number,
- payload length,
- CRC,
- explicit error handling.

The existing InstrumentProtocol v1.0 should be audited and reused where compatible.

Do not silently create competing protocol definitions.

---

11. LOGIC ANALYZER

Implement a real hardware-assisted logic analyzer.

Target:

```
8 channels
CH0 = RP2040 GP2
CH1 = RP2040 GP3
CH2 = RP2040 GP4
CH3 = RP2040 GP5
CH4 = RP2040 GP6
CH5 = RP2040 GP7
CH6 = RP2040 GP8
CH7 = RP2040 GP9
```

Subject to physical verification.

Use:

```
GPIO
 ↓
PIO
 ↓
DMA
 ↓
SRAM
```

Do not implement high-speed acquisition with a normal polling loop.

---

12. 64-KIB CIRCULAR CAPTURE

Maintain approximately:

```
64 KiB
```

of digital sample storage.

One sample represents the state of eight channels:

```
1 sample = 1 byte
```

Therefore:

```
65,536 samples
```

Capture duration:

```
capture_time = 65,536 / sample_rate
```

The capture engine must support:

```
continuous acquisition
        ↓
circular DMA
        ↓
```

```
trigger
        ↓
preserve pre-trigger samples
        ↓
post-trigger acquisition
        ↓
freeze
        ↓
transfer
```

Default:

```
70% pre-trigger
30% post-trigger
```

Make the ratio configurable internally.

---

13. HARDWARE TRIGGER ENGINE

Implement:

Edge

```
rising
falling
either
```

Level

```
high
low
```

Pattern

8-bit pattern + mask.

Example:

```
pattern = 1010xxxx
mask    = 11110000
```

Glitch

Detect pulses shorter than a specified threshold.

Runt

Detect pulses that cross a threshold but fail duration requirements.

External trigger

```
RP2040 GP10
```

Trigger position

Default:

```
70% pre
30% post
```

---

14. TRIGGER HONESTY

Every trigger must be classified as:

```
HARDWARE DETERMINISTIC
PIO-ASSISTED
DMA-ASSISTED
FIRMWARE REAL-TIME
POST-PROCESSING
```

Never call software analysis a hardware trigger.

If the acquisition records continuously and firmware searches the completed buffer afterward, that is:

```
POST-PROCESSING
```

not hardware triggering.

---

15. DIGITAL MEASUREMENTS

Implement where hardware supports them:

- frequency,
- period,
- duty cycle,
- pulse width,
- rising-edge count,
- falling-edge count,
- event interval,
- timestamped edges.

Prefer:

- PIO,
- PWM counters,
- hardware timers,
- DMA,

over software polling.

Document actual tested limits.

Do not advertise theoretical MCU limits as guaranteed instrument performance.

---

16. ADC MEASUREMENT

Target:

```
GP26 = ADC0
GP27 = ADC1
GP28 = ADC2
```

Subject to hardware verification.

The measurement system must account for:

- reference,
- input scaling,
- protection,
- calibration,
- ADC resolution,
- conversion timing,
- averaging,
- filtering.

Never display raw ADC counts as volts without applying the actual hardware transfer function.

---

17. EXTERNAL PRESCALER

Target:

```
GP14
```

Control external hardware supporting:

```
1:1
1:2
1:4
```

Frequency correction:

```
actual_frequency =
measured_frequency × prescaler_ratio
```

The UI must indicate when a prescaler is active.

---

18. PROBE / INPUT PROTECTION

The firmware must never assume that RP2040 GPIOs are safe to connect directly to arbitrary electronics.

Conceptual path:

```
External signal
      ↓
ESD protection
      ↓
overvoltage protection
      ↓
current limiting
      ↓
signal conditioning
      ↓
RP2040
```

The exact protection circuit is a hardware design decision.

Document:

- maximum safe input,
- negative-voltage tolerance,
- ESD behavior,
- threshold,
- input impedance,
- attenuation,
- protection components.

---

19. USB SUBSYSTEM

USB becomes a first-class subsystem.

The project must distinguish:

```
USB HOST
USB DEVICE
USB CONTROL/PROTOCOL ANALYSIS
USB LOW-SPEED SNIFFING
```

```
USB FULL-SPEED SNIFFING
USB HIGH-SPEED LIMITATION
```

Do not combine these into one vague "USB analyzer" feature.

---

20. USB HOST MODE

Implement USB Host functionality where the physical USB interface supports it.

The host subsystem should support:

```
device attach
device detach
bus reset
enumeration
descriptor retrieval
device identification
configuration
interface discovery
endpoint discovery
control transfers
```

Decode at minimum:

```
Device Descriptor
Configuration Descriptor
Interface Descriptor
Endpoint Descriptor
String Descriptor
Interface Association Descriptor
BOS Descriptor where supported
```

The system should report:

```
VID
PID
bcdUSB
device class
subclass
protocol
manufacturer
product
serial
configuration
interfaces
endpoints
```

---

21. USB STANDARD REQUEST DECODER

Decode standard control requests including:

```
GET_DESCRIPTOR
```

```
SET_ADDRESS
SET_CONFIGURATION
GET_CONFIGURATION
GET_INTERFACE
SET_INTERFACE
GET_STATUS
CLEAR_FEATURE
SET_FEATURE
```

Represent control transactions structurally:

```
SETUP
bmRequestType
bRequest
wValue
wIndex
wLength

DATA
STATUS
```

Do not merely display raw hexadecimal bytes.

---

22. USB DEVICE MODE

Where supported by the selected USB hardware path, implement USB Device functionality.

The UDOO instrument should be capable of presenting itself as configurable test devices.

Initial device classes may include:

```
CDC
HID
vendor-specific diagnostic device
```

Potential future classes:

```
MSC
composite device
```

The purpose is testing and development, not pretending to be a complete commercial USB compliance tester.

---

23. USB DEVICE TESTING

Create controlled test functions:

```
usb device start
usb device stop
usb device identity
usb device enumerate
usb device control-test
```

```
usb device hid-test
usb device cdc-test
```

Record:

- host requests,
- enumeration sequence,
- control requests,
- errors,
- timing,
- resets.

---

24. USB LOW/FULL-SPEED SNIFFER

Implement a genuine low/full-speed USB bus-analysis architecture only after the physical electrical interface has been verified.

Do not connect USB D+/D− directly to RP2040 GPIO without suitable electrical consideration.

The intended architecture is:

```
USB D+
USB D-
   │
   ▼
Differential receiver / USB front-end
   │
   ▼
RP2040
   │
   ├── PIO
   ├── DMA
   └── timestamping
   │
   ▼
USB packet decoder
   │
   ▼
ESP32
   │
   ▼
Desktop
```

The exact front-end must be based on an actual tested circuit.

---

25. USB SNIFFER PHYSICAL REQUIREMENT

The UDOO KEY's onboard USB-C connector must not automatically be treated as an arbitrary USB bus-sniffing connection.

The USB-C connector has existing board-level purposes and must be audited.

Before implementing sniffing, determine:

```
USB-C D+
USB-C D-
USB role
USB routing
JP1 state
MCU connection
ESD/protection
host/device capability
physical accessibility
```

If arbitrary external USB bus capture requires an external tap/front-end, document that explicitly.

Do not pretend the onboard USB connector alone provides a passive USB analyzer.

---

26. USB PACKET DECODER

Implement the low/full-speed decoding pipeline conceptually as:

```
electrical differential state
        ↓
line state
        ↓
NRZI
        ↓
bit unstuffing
        ↓
PID
        ↓
token/data/handshake
        ↓
CRC
        ↓
packet
        ↓
transaction
        ↓
transfer
```

Decode:

```
SYNC
PID
ADDR
ENDPOINT
```

```
DATA
CRC
EOP
```

Represent errors explicitly:

```
CRC_ERROR
BIT_STUFF_ERROR
INVALID_PID
SYNC_ERROR
TIMEOUT
UNEXPECTED_EOP
UNKNOWN_PACKET
```

---

27. USB TRANSACTION MODEL

The analyzer should not stop at individual packets.

Build:

```
packet
 ↓
transaction
 ↓
transfer
 ↓
control request
```

Example:

```
SETUP
DATA
STATUS
```

becomes one logical transaction.

The desktop should display both:

```
raw packets
```

and:

```
decoded USB transaction
```

---

28. USB TRIGGERS

Reuse the existing capture-trigger architecture.

Possible USB triggers:

```
SETUP PID
IN PID
```

```
OUT PID
DATA0
DATA1
specific address
specific endpoint
specific PID
specific payload
descriptor request
VID/PID-related event
CRC error
bus reset
```

Support:

```
pre-trigger
trigger event
post-trigger
```

where physically achievable.

---

29. USB HIGH-SPEED LIMITATION

Do not claim that the UDOO KEY can passively capture arbitrary USB 2.0 high-speed traffic at 480 Mbps merely because the MCU is fast.

Treat:

```
USB Low Speed
USB Full Speed
USB High Speed
```

as separate engineering problems.

Low/full-speed sniffing may be a realistic RP2040/PIO project with appropriate electrical front-end hardware.

480-Mbps high-speed capture should be treated as a future dedicated hardware architecture involving appropriate USB high-speed PHY/FPGA or equivalent acquisition hardware.

---

30. USB ANALYSIS UI

Create a USB analyzer workspace.

It should show:

```
USB DEVICE
──────────────────
VID:PID
Manufacturer
Product
Serial
Class
```

```
ENUMERATION
──────────────────────
RESET
GET_DESCRIPTOR
SET_ADDRESS
GET_DESCRIPTOR
SET_CONFIGURATION


PACKETS
──────────────────────
12.431 ms  SETUP
12.438 ms  DATA0
12.442 ms  ACK


TRANSACTIONS
──────────────────────
GET_DESCRIPTOR(Device)
Address 0
Length 18
```

Allow switching between:

```
Decoded
Raw
Timing
Waveform
Errors
```

---

31. GENERIC PACKET / CAPTURE ARCHITECTURE

USB should use the same general capture framework as:

- Wi-Fi,
- BLE,
- UART,
- SPI,
- I2C,
- IR,
- nRF24,
- logic capture.

Define a generic internal representation:

```
CaptureSource
Timestamp
Channel
Direction
Length
RawData
DecodedData
Flags
Errors
```

Do not force all protocols into an identical wire format.

Use protocol-specific metadata layered over a common capture model.

---

32. ESP32 WI-FI

The ESP32 subsystem should provide real generic Wi-Fi functionality.

Implement:

```
Wi-Fi status
scan
connect
disconnect
RSSI
channel
IP configuration
gateway
DNS
latency
connectivity diagnostics
```

Where the ESP32 hardware/SDK supports it, provide bounded Wi-Fi packet capture/promiscuous analysis.

Never claim arbitrary monitor-mode capability beyond what the ESP32 actually provides.

---

33. WI-FI PACKET DECODING

Decode generic packet layers where data is available:

```
802.11
Ethernet
ARP
IPv4
IPv6
ICMP
UDP
TCP
DNS
DHCP
```

Use a generic packet record.

Support export to:

```
PCAP
PCAPNG
```

only where the capture data genuinely maps to those formats.

Do not fabricate protocol headers merely to create a PCAP file.

---

34. BLUETOOTH / BLE

Implement generic Bluetooth functionality.

BLE:

```
scan
advertising capture
RSSI
MAC/address
advertising data
service UUIDs
manufacturer data
```

Bluetooth Classic:

```
discovery
device identification
connection diagnostics
```

only where supported by the actual ESP32 Bluetooth stack and firmware.

Do not claim arbitrary Bluetooth packet sniffing if the hardware cannot provide it.

---

35. IR

Retain the existing IR subsystem.

Target:

```
ESP32 RMT TX
ESP32 RMT RX
raw pulse capture
protocol decoding
```

Existing GPIO/channel assignments must be audited against the actual hardware.

Support:

```
raw capture
send raw
decode known protocols
save capture
```

```
replay capture
```

---

36. NRF24

Retain the nRF24 subsystem as an external RF peripheral.

Existing UEXT resources must be managed through the ESP32 resource manager.

Do not allow:

```
nRF24
UEXT
inter-MCU SPI
```

to silently conflict.

Use explicit resource locking.

---

37. RESOURCE MANAGER

Create a central ESP32 resource manager.

It owns:

```
GPIO
UART
SPI
I2C
Wi-Fi
Bluetooth
BLE
IR
nRF24
external instrument transport
```

The manager must detect conflicts before enabling a subsystem.

Example:

```
REQUEST GPIO15
       ↓
RESOURCE MANAGER
       ↓
nRF24 owns GPIO15
       ↓
REQUEST DENIED
```

Never silently steal a resource from another subsystem.

---

38. BUS-PIRATE-STYLE TERMINAL

Provide a real terminal interface.

The terminal must not be Apple-specific.

Core commands should include concepts such as:

```
help
version
status
pins
gpio
uart
spi
i2c
1wire
logic
measure
adc
trigger
usb
wifi
ble
bt
ir
nrf24
capture
scope
diagnostic
selftest
```

Use mode prompts where useful:

```
>
LOGIC>
USB>
UART>
SPI>
I2C>
WIFI>
BLE>
SCOPE>
```

The C++ application terminal and any direct terminal must exercise the same command layer.

Do not maintain separate incompatible terminal implementations.

---

39. EXTERNAL INSTRUMENT INTEGRATION

The UDOO KEY should support a generic:

```
External Instrument Control
```

subsystem.

Target interfaces may include:

```
RS-232
HP-IB / GPIB
SCPI
USB instruments
network instruments
```

Only implement interfaces for which actual physical hardware is available.

---

40. HP 54520A INTEGRATION

The HP 54520A is a target external instrument.

The objective is **not to replace the oscilloscope**.

The objective is to make the UDOO KEY a:

```
controller
communications bridge
measurement coordinator
trigger coordinator
waveform acquisition bridge
protocol correlation engine
```

for the HP 54520A.

The HP documentation must be treated as the command/protocol authority.

---

41. HP 54520A HARDWARE AUDIT

Before implementing the interface, inspect the actual oscilloscope.

Determine:

```
exact model
firmware version
installed interface hardware
HP-IB availability
RS-232 availability
connector type
address configuration
serial configuration
available command interface
```

Do not assume a particular interface exists on every unit.

Record:

```
ANSWER
SOURCE
CONFIDENCE
```

```
ACTION
```

---

42. HP-IB / GPIB

If the 54520A has HP-IB/GPIB available, evaluate it as the preferred legacy-instrument control path.

Potential architecture:

```
HP 54520A
     |
    HP-IB
     |
GPIB interface hardware
     |
   ESP32
     |
InstrumentProtocol
     |
UDOO KEY
     |
C++ application
```

The UDOO KEY must not directly connect ESP32 GPIOs to HP-IB without an electrically appropriate interface.

HP-IB requires suitable electrical transceivers/interface hardware.

---

43. RS-232

If the 54520A has an available RS-232 interface:

```
HP 54520A
     |
   RS-232
     |
RS-232 transceiver
     |
ESP32
```

Never connect RS-232 voltage levels directly to ESP32 or RP2040 GPIO.

Use an appropriate level-shifting/transceiver circuit.

---

44. HP RAW COMMAND MODE

The desktop application should provide a transparent command path.

Concept:

```
scope raw <command>
```

The raw command is passed to the instrument without the UDOO attempting to reinterpret it.

This provides a debugging escape hatch and allows testing commands before higher-level wrappers exist.

---

45. HP NORMALIZED COMMAND API

Build higher-level commands around the actual 54520A command set.

Example conceptual interface:

```
scope connect
scope disconnect
scope identify
scope status

scope run
scope stop
scope single

scope channel 1
scope channel 2

scope timebase
scope trigger

scope measure
scope waveform
```

Do not invent command syntax that the HP itself does not support internally.

The UDOO API is an abstraction layer over the actual instrument protocol.

---

46. HP WAVEFORM ACQUISITION

Investigate the actual 54520A waveform-transfer commands.

Implement:

```
request waveform
receive waveform
validate response
decode format
convert to engineering units
attach timestamp
store capture
```

The application should retain:

```
raw instrument response
```

as well as:

```
decoded waveform
```

for troubleshooting.

Do not throw away raw acquisition data before validation.

---

47. HP WAVEFORM DATA MODEL

Create a generic waveform model:

```
Waveform
  ├── source
  ├── channel
  ├── timestamp
  ├── sample_count
  ├── sample_interval
  ├── vertical_scale
  ├── vertical_offset
  ├── units
  ├── samples
  └── acquisition_metadata
```

This allows the same desktop application to eventually display:

```
UDOO capture
HP capture
mixed capture
```

---

48. CROSS-INSTRUMENT CORRELATION

This is a major project feature.

The UDOO digital analyzer and external oscilloscope should be treated as complementary instruments.

Example:

```
USB bus
  │
  ├──────────────► UDOO USB analyzer
  │
  └──────────────► HP 54520A
```

Or:

```
SPI
  │
  ├──────────────► RP2040 logic analyzer
  │
  └──────────────► HP analog channel
```

The desktop should eventually correlate:

```
digital protocol event
        +
analog waveform
        +
timestamp
```

---

49. CROSS-TRIGGER ARCHITECTURE

Do not initially promise electrical trigger synchronization.

First implement software-assisted coordination:

```
UDOO detects event
       ↓
timestamp event
       ↓
send command to HP
       ↓
HP acquires
       ↓
return waveform
       ↓
correlate in desktop
```

Later, investigate a physical trigger line if the HP and UDOO interfaces support it.

If a physical trigger is implemented:

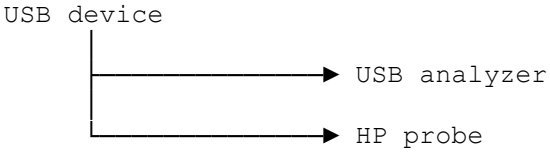
```
RP2040 trigger output
        ↓
HP external trigger input
```

must be electrically verified.

---

50. EXAMPLE USB + HP ANALYSIS

Target workflow:



The UDOO detects:

```
SETUP
Address = 4
Endpoint = 0
GET_DESCRIPTOR
```

The desktop displays:

```
USB EVENT
─────────────────────────

GET_DESCRIPTOR
Address: 4
Endpoint: 0
Length: 18

Timestamp:
12.483921 ms
```

Then display the corresponding HP waveform:

```
HP 54520A CH1

      waveform
──────┐    ┌──────
      └────┘

         ↑
      correlated
      USB event
```

This is the long-term target.

---

51. DESKTOP APPLICATION ARCHITECTURE

The C++ desktop application becomes the central analysis environment.

Target workspaces:

```
Terminal
Logic Analyzer
USB Analyzer
UART Analyzer
SPI Analyzer
I2C Analyzer
JTAG/SWD
Wi-Fi
Bluetooth/BLE
IR
nRF24
Measurements
External Instruments
Oscilloscope
Diagnostics
```

The desktop must not contain separate incompatible protocol implementations.

Use the same:

```
InstrumentProtocol
CaptureModel
```

```
TimestampModel
DeviceManager
CommandDispatcher
```

where practical.

---

52. GENERIC CAPTURE MODEL

Create a common capture representation.

Conceptually:

```
Capture
 ├── source
 ├── timestamp
 ├── duration
 ├── sample_rate
 ├── channel_count
 ├── raw_data
 ├── decoded_events
 ├── trigger_metadata
 ├── errors
 └── annotations
```

Protocol-specific extensions should exist for:

```
USB
SPI
I2C
UART
Wi-Fi
BLE
IR
nRF24
Logic
```

---

53. TIMESTAMP ARCHITECTURE

Timestamping must be explicit.

Sources may include:

```
RP2040 hardware timestamp
ESP32 timestamp
desktop timestamp
HP waveform timestamp
```

Never silently compare timestamps from different clocks.

Maintain synchronization metadata.

Where exact synchronization is unavailable, label correlation as:

```
approximate
software-correlated
hardware-synchronized
```

Do not call software correlation hardware synchronization.

---

54. APPLE-SPECIFIC FUNCTIONALITY

The project must contain **zero Apple-specific functionality unless a future requirement explicitly adds it back**.

Remove:

```
AppleWorkspace
AppleHardwareBridge
AppleCapabilityManager
Apple USB analysis
Apple Lightning-specific decoders
Apple-specific Bluetooth logic
Apple-specific Wi-Fi diagnostics
Apple-specific reporting
Apple-specific automation
Apple-specific test fixtures
```

Do not remove generic functionality merely because it was originally used for Apple analysis.

For example:

```
generic USB descriptor decoder
generic USB capture
generic BLE scanner
generic Wi-Fi packet decoder
generic PCAP
generic USB control transfer decoder
```

must remain.

The objective is:

> generic real electronics functionality.

Not:

> Apple functionality renamed to generic functionality.

---

55. NO FAKE RADIO CAPABILITIES

Distinguish:

```
implemented
```

from:

```
serialization test
```

and:

```
hardware abstraction
```

and:

```
real radio capture
```

For every Wi-Fi/BLE/Bluetooth subsystem record:

```
UNIT TEST
INTEGRATION TEST
HARDWARE TEST
```

Do not claim radio hardware functionality merely because a test passes.

---

56. JTAG / SWD

Retain generic JTAG/SWD support.

Audit:

- physical pins,
- voltage,
- direction,
- reset,
- debug ownership,
- connector availability.

The debug engine must distinguish:

```
JTAG
SWD
```

and not silently substitute one for the other.

---

57. BUS-PIRATE REFERENCE SOURCES

The project contains:

- Bus Pirate 5 source,
- ESP32 Bus Pirate source.

Treat these as **engineering references**, not source code to blindly copy.

Before implementing a feature:

1. search the Bus Pirate references,
2. identify relevant implementation,
3. identify hardware assumptions,
4. determine whether UDOO KEY can support it,
5. determine whether RP2040 hardware improves it,
6. adapt only the useful behavior.

Relationship:

```
Bus Pirate
     ↓
functional reference
     ↓
hardware/resource analysis
     ↓
UDOO KEY implementation
```

Never:

```
Bus Pirate
     ↓
copy/paste
     ↓
UDOO KEY
```

---

58. BUS-PIRATE FEATURE MATRIX

Create:

```
docs/BUS_PIRATE_REFERENCE_MAP.md
```

with:

```
Feature
Bus Pirate implementation
ESP32 implementation
UDOO equivalent
MCU owner
Reuse / Adapt / Rewrite / Defer
Hardware assumptions
Reason
Status
```

Priority:

First

Reuse proven algorithms.

Second

Reuse behavioral models.

Third

Reuse user-interface concepts.

Fourth

Adapt hardware abstractions.

Last

Port MCU-specific implementation.

---

59. REFERENCE CODE DOES NOT OVERRIDE HARDWARE TRUTH

Reference code must never override:

1. UDOO schematic,
2. official board documentation,
3. verified physical board,
4. this resource ownership model,
5. actual hardware tests.

---

60. SOURCE AUDIT

Before modifying the repository, search for:

```
Bus Pirate
ESP32 Bus Pirate
logic analyzer
USB
Wi-Fi
BLE
Bluetooth
PCAP
IR
nRF24
JTAG
SWD
Apple
Lightning
scope
```

```
54520
GPIB
HP-IB
RS-232
SCPI
ST7735
PIO
DMA
UART
SPI
I2C
ADC
trigger
capture
```

Identify:

```
implemented
partial
dead
duplicate
conflicting
reference-only
missing
```

---

61. HARDWARE RESOURCE MANAGER

Create one authoritative resource map.

Resources include:

```
RP2040 GPIO
ESP32 GPIO
PIO state machines
DMA channels
UART
SPI
I2C
ADC
timers
display
buttons
USB
Wi-Fi
BLE
Bluetooth
IR RMT
nRF24
internal UART
external instrument transport
```

Every subsystem must declare what it owns.

---

62. BUILD SYSTEM

Maintain independent build targets:

```
rp2040_logic_analyzer
esp32_coprocessor
udoo-key-app
```

Potential test targets:

```
native unit tests
protocol tests
decoder tests
simulation tests
```

Do not confuse native tests with hardware tests.

---

63. TEST CLASSIFICATION

Every test must be labelled:

```
UNIT
INTEGRATION
SIMULATION
FLASHED
HARDWARE
EXTERNAL-INSTRUMENT
```

Example:

```
USB CRC decoder
= UNIT

USB RP2040 capture
= HARDWARE

HP command parser
= UNIT

HP 54520A command exchange
= EXTERNAL-INSTRUMENT

Wi-Fi parser
= UNIT

Wi-Fi scan
= HARDWARE
```

---

64. HARDWARE DIAGNOSTIC MODE

Create a dedicated hardware diagnostic mode.

Test:

```
RP2040
ESP32
internal UART
display
buttons
GPIO
ADC
PIO
DMA
trigger
USB
Wi-Fi
BLE
IR
nRF24
external instrument transport
```

Display:

```
UDOO KEY DIAGNOSTICS

RP2040        OK
ESP32         OK
UART          OK
DISPLAY       OK
BUTTONS       OK
ADC           OK
PIO           OK
DMA           OK
TRIGGER       OK
USB           ?
WIFI          OK
BLE           OK
IR            ?
NRF24         ?
HP INTERFACE  ?
```

Use:

```
OK
FAIL
NOT PRESENT
NOT TESTED
REQUIRES HARDWARE
```

Never use `OK` merely because initialization succeeded.

---

65. USB HARDWARE DIAGNOSTIC

USB diagnostics must separately report:

```
USB interface detected
USB electrical interface available
host mode available
device mode available
low-speed sniffing available
full-speed sniffing available
high-speed sniffing unavailable
```

Do not collapse these into one `USB: OK`.

---

66. HP HARDWARE DIAGNOSTIC

Provide:

```
scope status
scope identify
scope interface
scope raw
```

Example:

```
EXTERNAL INSTRUMENT

HP 54520A
Interface: HP-IB
Connection: ONLINE
Firmware: detected
Command test: PASS
Waveform query: PASS
```

If the interface is absent:

```
HP 54520A
Interface: NOT DETECTED
```

---

67. BUILD VALIDATION

After every meaningful modification:

1. build RP2040,
2. build ESP32,
3. build desktop,
4. run affected tests,
5. inspect warnings,

6. inspect protocol compatibility,
7. inspect resource conflicts,
8. document result.

Do not accumulate huge batches of untested modifications.

---

68. NO FAKE COMPLETION

Never say:

```
implemented
verified
tested
working
```

unless the corresponding action actually occurred.

Use:

```
DESIGNED
IMPLEMENTED
COMPILED
FLASHED
SIMULATION TESTED
HARDWARE TESTED
EXTERNAL HARDWARE TESTED
VERIFIED
```

These are separate states.

---

69. NO ARCHITECTURE DRIFT

Do not introduce:

- another MCU,
- unnecessary FPGA,
- unnecessary external RAM,
- unnecessary SPI transport,
- invented GPIOs,
- invented peripherals,
- invented USB capabilities,
- invented HP interfaces,
- unnecessary RTOS complexity,

- unnecessary abstraction layers.

Exceptions require evidence.

If a major architectural change is required, document:

```
Existing limitation
Evidence
Proposed change
Alternatives
Benefits
Risks
Hardware impact
Software impact
```

Then stop before making a material architecture change unless explicitly authorized.

---

70. HARDWARE FREEZE PROCESS

Before declaring hardware frozen:

Resolve:

```
board revision
RP2040 GPIO map
ESP32 GPIO map
internal UART
display
buttons
ADC
trigger
prescaler
SWD
USB
UEXT
IR
nRF24
probe interface
HP interface
```

For each:

```
ANSWER
SOURCE
CONFIDENCE
ACTION
```

---

71. DEFINITIVE HARDWARE MAP

Produce:

MCU	GPIO	Physical Location	Electrical Function	Project Function	Owner	Status
RP2040	GP0	internal	UART	ESP32 transport	Transport	VERIFIED
RP2040	GP1	internal	UART	ESP32 transport	Transport	VERIFIED
RP2040	GP2	?	GPIO	Logic CH0	Logic	?
RP2040	GP3	?	GPIO	Logic CH1	Logic	?
RP2040	GP4	?	GPIO	Logic CH2	Logic	?
RP2040	GP5	?	GPIO	Logic CH3	Logic	?
RP2040	GP6	?	GPIO	Logic CH4	Logic	?
RP2040	GP7	?	GPIO	Logic CH5	Logic	?
RP2040	GP8	?	GPIO	Logic CH6	Logic	?
RP2040	GP9	?	GPIO	Logic CH7	Logic	?
RP2040	GP10	?	GPIO	External Trigger	Trigger	?
RP2040	GP11	?	GPIO	TFT Reset	Display	?
RP2040	GP12	?	GPIO	TFT DC	Display	?
RP2040	GP13	?	GPIO	TFT CS	Display	?
RP2040	GP14	?	GPIO	Prescaler	Measurement	?
RP2040	GP25	onboard	LED	Green LED	Board	VERIFIED
RP2040	GP26	?	ADC	ADC0	Measurement	?
RP2040	GP27	?	ADC	ADC1	Measurement	?
RP2040	GP28	?	ADC	ADC2	Measurement	?

This becomes the **single hardware truth**.

---

72. IMPLEMENTATION WORKFLOW

Follow this exact process.

Phase 1 — Repository and hardware audit

Do not change architecture.

Determine:

```
what exists
what works
what conflicts
what is missing
what is fake/mock
what requires hardware
```

Produce:

```
Implementation Gap Report
```

with:

```
CRITICAL BLOCKERS
```

```
HIGH PRIORITY
MEDIUM PRIORITY
ALREADY COMPLETE
```

---

73. PHASE 2 — HARDWARE FREEZE

Resolve the physical board.

Verify:

```
GPIO
UART
display
buttons
ADC
trigger
prescaler
USB
UEXT
SWD
```

Freeze verified resources.

---

74. PHASE 3 — MCU BASELINE

Independently prove:

```
RP2040 boots
ESP32 boots
internal UART works
display works
buttons work
```

---

75. PHASE 4 — RP2040 ACQUISITION

Implement and verify:

```
GPIO
 ↓
PIO
 ↓
DMA
 ↓
RAM
```

before implementing sophisticated UI.

---

76. PHASE 5 — CIRCULAR CAPTURE

Implement:

```
64 KiB circular DMA
```

Prove:

- continuous acquisition,
- wraparound,
- no corruption,
- deterministic stopping,
- metadata correctness.

---

77. PHASE 6 — TRIGGERS

Implement sequentially:

```
edge
level
pattern
external
glitch
runt
```

Test each independently.

---

78. PHASE 7 — MEASUREMENTS

Implement:

```
frequency
period
duty cycle
pulse width
edge count
ADC
prescaler correction
```

---

79. PHASE 8 — INTERNAL TRANSPORT

Prove:

```
PING
```

```
STATUS
CONFIG
CAPTURE INFO
CAPTURE DATA
```

Then stress-test:

```
throughput
CRC
sequence handling
packet loss
buffering
transfer time
```

---

80. PHASE 9 — USB

Implement in this order:

1

USB descriptors and host enumeration.

2

USB control-transfer decoding.

3

USB device-mode test functionality.

4

Low/full-speed physical sniffer architecture.

5

PIO/DMA packet acquisition.

6

USB packet decoder.

7

USB transaction decoder.

8

USB triggers.

9

Desktop USB analyzer.

10

Waveform correlation.

Do not start with the sniffer before proving the physical interface.

---

81. PHASE 10 — ESP32 NETWORK/RF

Implement and hardware-test:

```
Wi-Fi scan
Wi-Fi diagnostics
Wi-Fi packet capture where supported
packet decoding
PCAP export
BLE scanning
Bluetooth discovery
IR
nRF24
```

---

82. PHASE 11 — EXTERNAL INSTRUMENTS

Implement:

```
instrument discovery
raw transport
instrument identification
command/query
measurement
waveform acquisition
```

Start with:

```
HP 54520A
```

if the physical interface is available.

---

83. PHASE 12 — DESKTOP UI

Only after the underlying engines work.

Implement:

```
Terminal
```

```
Logic Analyzer
USB Analyzer
Network Analyzer
Bluetooth
Measurements
External Instruments
Oscilloscope
Diagnostics
```

---

84. PHASE 13 — CROSS-CORRELATION

Implement:

```
protocol event
+
timestamp
+
analog waveform
```

Support:

```
USB + HP
SPI + HP
UART + HP
logic + HP
trigger + HP
```

where physically achievable.

---

85. PHASE 14 — FINAL HARDWARE TEST

Use real signals.

At minimum test:

Logic analyzer

Known square wave.

Frequency

Known frequency source.

Trigger

Known edge/pattern.

ADC

Known DC voltages within safe limits.

UART

Known serial stream.

SPI

Known SPI transaction.

I2C

Known I2C transaction.

USB

Known USB device.

Wi-Fi

Known access point.

BLE

Known advertising device.

IR

Known transmitter.

nRF24

Known compatible radio.

HP

Actual HP 54520A if available.

---

86. STATUS TABLE

Maintain a living status document.

Subsystem	Designed	Implemented	Compiled	Hardware Tested	Status
RP2040 boot		✓			
ESP32 boot		✓			

Subsystem	Designed	Implemented	Compiled	Hardware Tested	Status
Internal UART	✓				
Display	✓				
Buttons	✓				
8ch acquisition	✓				
PIO	✓				
DMA	✓				
Circular capture	✓				
Trigger	✓				
ADC	✓				
Measurements	✓				
USB Host	✓				
USB Device	✓				
USB LS sniffer	✓				
USB FS sniffer	✓				
Wi-Fi	✓				
BLE	✓				
Bluetooth	✓				
IR	✓				
nRF24	✓				
JTAG/SWD	✓				
HP 54520A	✓				
Desktop terminal	✓				
Desktop analyzer	✓				
Cross-correlation	✓				

---

87. DOCUMENTATION REQUIREMENTS

Maintain:

```
docs/
    HARDWARE_MAP.md
    ARCHITECTURE.md
    IMPLEMENTATION_STATUS.md
    TEST_PLAN.md
    USB_ARCHITECTURE.md
    BUS_PIRATE_REFERENCE_MAP.md
    EXTERNAL_INSTRUMENTS.md
    HP54520A_INTEGRATION.md
    PROTOCOL.md
    RESOURCE_MAP.md
```

Every major subsystem must have:

```
architecture
```

```
hardware requirements
software implementation
commands
test procedure
known limitations
status
```

---

88. USB DOCUMENTATION REQUIREMENTS

`USB_ARCHITECTURE.md` must explicitly document:

```
USB physical topology
host mode
device mode
descriptor decoder
control transfer decoder
LS capture
FS capture
packet decoder
transaction decoder
triggering
electrical front-end
limitations
high-speed limitations
hardware test procedure
```

---

89. HP DOCUMENTATION REQUIREMENTS

`HP54520A_INTEGRATION.md` must document:

```
instrument model
firmware
physical interface
interface hardware
command protocol
connection procedure
identification
raw commands
waveform acquisition
waveform format
error handling
timing
trigger coordination
timestamp correlation
known limitations
```

---

90. TERMINAL REQUIREMENTS

Every major subsystem should have a terminal interface.

Examples:

```
logic status
logic start
logic stop
logic capture

usb scan
usb info
usb host start
usb device start
usb sniff start
usb sniff stop

wifi scan
wifi status
wifi capture

ble scan

scope list
scope connect
scope identify
scope run
scope stop
scope single
scope waveform
scope raw
```

The terminal should be useful without the graphical application.

---

91. C++ APPLICATION REQUIREMENTS

The C++ application must provide:

```
device connection
terminal
capture control
waveform visualization
protocol decoding
USB analysis
network analysis
external instrument control
HP waveform visualization
diagnostics
export
logging
```

The application must not contain Apple-specific assumptions.

---

92. ERROR HANDLING

Every subsystem must fail explicitly.

Examples:

```
USB_NO_DEVICE
USB_CRC_ERROR
USB_BIT_STUFF_ERROR
USB_TIMEOUT

WIFI_NOT_INITIALIZED
WIFI_CAPTURE_UNAVAILABLE

BLE_UNSUPPORTED

HP_NOT_CONNECTED
HP_INTERFACE_UNAVAILABLE
HP_COMMAND_ERROR
HP_WAVEFORM_ERROR

CAPTURE_OVERFLOW
DMA_ERROR
UART_CRC_ERROR
RESOURCE_BUSY
INVALID_CONFIGURATION
```

Never silently fail.

---

93. PERFORMANCE RULE

Measure before optimizing.

Record:

```
sampling rate
capture depth
DMA throughput
UART throughput
capture transfer time
USB decoder throughput
Wi-Fi packet throughput
BLE event rate
HP waveform transfer time
desktop processing time
```

Optimize actual bottlenecks.

Do not redesign architecture based on theoretical performance.

---

94. HARDWARE-SAFETY RULE

Never test unknown external signals directly on an RP2040 GPIO.

Before hardware testing determine:

```
voltage
current
polarity
frequency
edge rate
ground relationship
```

Use appropriate:

```
protection
attenuation
level shifting
isolation
probe hardware
```

when necessary.

---

95. IMPORTANT UDOO KEY WARNINGS

Do not confuse:

```
RP2040 GP19
```

with:

```
ESP32 GPIO19
```

Always identify the MCU.

Do not repurpose:

```
RP2040 GP0
RP2040 GP1
```

without proving that the internal transport is no longer required.

Treat:

```
RP2040 GP25
```

as the onboard green LED resource.

Do not treat ESP32 UEXT resources as RP2040 GPIOs.

Do not apply UDOO KEY Pro-only peripherals to a Basic board.

---

96. CURRENT HARDWARE QUESTIONS

Close all of these explicitly:

RP2040

- GP2 physical accessibility
- GP3 physical accessibility
- GP4–GP9 physical accessibility
- GP10 physical accessibility
- GP11–GP22 physical accessibility
- GP23 ownership
- GP24 ownership
- GP26–GP28 ADC accessibility
- GP29 ownership

USB

- USB-C routing
- USB role
- JP1 behavior
- D+
- D-
- host capability
- device capability
- electrical access for sniffing
- required external USB front-end

HP

- exact 54520A variant
- installed interface
- HP-IB availability
- RS-232 availability
- interface electrical requirements
- command protocol
- waveform transfer support

Every question must have:

```
ANSWER
SOURCE
CONFIDENCE
ACTION
```

---

97. IMPLEMENTATION GAP REPORT

After the initial audit, produce:

CRITICAL BLOCKERS

Things preventing physical operation.

HIGH PRIORITY

Things required for major functionality.

MEDIUM PRIORITY

Important but non-blocking.

LOW PRIORITY

Polish and convenience.

ALREADY COMPLETE

Verified functionality.

HARDWARE REQUIRED

Features that cannot be completed without physical hardware.

---

98. EXECUTION RULE

From this point forward:

Step 1

Inspect the repository.

Step 2

Inspect hardware documentation.

Step 3

Inspect existing implementation.

Step 4

Compare against this specification.

Step 5

Produce the gap report.

Step 6

Fix the highest-priority real blocker.

Step 7

Build.

Step 8

Test.

Step 9

Document.

Step 10

Move to the next blocker.

Do not restart the architecture discussion unless new physical evidence contradicts the current architecture.

---

99. DEFINITION OF DONE

The project is not complete because:

- the architecture looks good,
- the code compiles,

- the UI looks good,
- tests are mocked,
- or Gemini says it is complete.

The instrument is complete when the corresponding capabilities have been demonstrated.

Minimum core completion:

1. both MCUs boot,
2. internal UART works,
3. displays operate,
4. buttons operate,
5. eight digital channels can be captured,
6. DMA operates continuously,
7. circular capture operates,
8. trigger works,
9. pre/post trigger works,
10. capture transfers correctly,
11. waveform is displayed,
12. measurements work,
13. ADC works,
14. prescaler correction works,
15. external trigger works,
16. errors are reported,
17. defined hardware tests pass.

Extended completion:

18. USB host works,
19. USB device mode works,
20. USB low-speed capture works if hardware supports it,
21. USB full-speed capture works if hardware supports it,
22. USB transactions decode,
23. Wi-Fi diagnostics work,
24. Wi-Fi packet capture works where supported,
25. BLE scanning works,
26. Bluetooth functions work where supported,
27. IR works,
28. nRF24 works,

29.JTAG/SWD functions work,

30.external instrument transport works,

31.HP 54520A integration works where the physical interface is available,

32.HP waveform acquisition works,

33.UDOO/HP timestamp correlation works,

34.C++ application provides unified analysis.

---

100. FINAL ENGINEERING PRINCIPLE

The goal is not to create the most complicated architecture.

The goal is to create the **most capable real instrument that can actually be built from the UDOO KEY hardware**.

The governing rule is:

```
VERIFY
   ↓
RECONCILE
   ↓
IMPLEMENT
   ↓
BUILD
   ↓
TEST
   ↓
HARDWARE VERIFY
   ↓
DOCUMENT
   ↓
FREEZE
```

The project must remain:

```
real
testable
deterministic
documented
hardware-aware
modular
honest about limitations
```

The UDOO KEY schematic and verified physical board override assumptions.

The RP2040 owns deterministic instrumentation.

The ESP32 owns communications and application services.

The USB subsystem provides host, device, and realistic low/full-speed analysis capabilities where the physical hardware permits.

The external-instrument subsystem allows the UDOO KEY to control and correlate measurements from instruments such as the HP 54520A.

Bus Pirate source provides proven engineering reference material.

The C++ desktop application provides the unified analysis environment.

No subsystem is considered complete until its actual implementation and corresponding test status are known.

Stop theorizing once sufficient evidence exists. Build the instrument.